HTML & CSS Learning Report

Practical Study Notes and Exercise-Based Learning

Focus: HTML structure, CSS styling, layout systems, and element positioning

Source basis: Uploaded exercise scan covering identifiable exercises from Lessons of *HTML & CSS Full Course — Beginner to Pro* by SuperSimpleDev

Purpose: Document the concepts reviewed, implementation patterns studied, and practical exercises to complete or verify.

> **Status note:** This report documents study material and the concepts covered in the accompanying notes. It does not claim that every exercise has been completed or independently verified. Update the progress section with your actual results before presenting this as a record of completed work.

1. Overview

This study focused on translating visual web-page requirements into HTML structures and CSS rules. The central workflow is to identify the elements shown in a reference design, group related elements, choose an appropriate layout system, style typography and spacing, and inspect the rendered page in a browser.

The material progresses from foundational styling and the CSS box model to display behavior, container-based layouts, CSS Grid, Flexbox, nested layouts, and fixed/absolute positioning. It also introduces browser developer tools as a practical way to inspect styles and debug visual mismatches.

2. Learning Objectives

- Understand the role of HTML in defining content and document structure.
- Write basic HTML documents and connect an external stylesheet.
- Apply CSS selectors and declarations to style elements.
- Distinguish width/height, padding, border, and margin.
- Style text, buttons, images, and input fields.
- Understand block and inline-block behavior.
- Use `div` elements to group content into reusable components.

- Build layouts with nested horizontal and vertical groups.
- Use CSS Grid to define columns and spacing.
- Use Flexbox to align items and distribute available space.
- Position elements using `fixed`, `relative`, and `absolute`.
- Use Chrome DevTools to inspect computed styles and diagnose styling issues.

3. HTML Fundamentals

HTML provides the structure and meaning of a page. Common elements in the exercises include headings, paragraphs, buttons, images, inputs, links, and generic containers.

Basic document structure

```
<!DOCTYPE html>
<html>
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Example Page</title>
 <link rel="stylesheet" href="styles.css">
</head>
<body>
 <h1>Page heading</h1>
 <p>Supporting text.</p>
 <button>Click me</button>
</body>
</html>
```

Concepts reviewed

- `<!DOCTYPE html>` declares the document as HTML5.
- `<head>` contains page metadata, the document title, and stylesheet links.
- `<body>` contains the visible page content.
- Elements use tags to define their role.
- Attributes provide additional information, such as an image `src` or an input `placeholder`.
- Nested elements should be indented clearly and closed in the correct order.

- A `class` attribute connects an HTML element to a reusable CSS class selector.
- A `div` groups related content when a more specific semantic element is not appropriate.

Example of HTML and CSS working together

```
<div class="notice">
 <p>Would you like to take our survey?</p>
 <button>Yes</button>
 <button>No</button>
</div>
```

```
.notice {
 border: 1px solid #ccc;
 padding: 16px;
 max-width: 360px;
}
```

The HTML defines the content and grouping; the CSS defines the border, internal spacing, and width constraint.

4. CSS Syntax, Selectors, and Styling

A CSS rule consists of a selector and a declaration block. Each declaration contains a property and a value.

```
.card {
 background-color: white;
 padding: 16px;
 border: 1px solid #ddd;
}
```

Selectors reviewed

Selector | Purpose

`p` | Selects paragraph elements

`.card` | Selects elements with class `card`

`#main` | Selects the element with ID `main`

`button:hover` | Applies styles while the pointer hovers over a button

`.card p` | Selects paragraphs nested inside an element with class `card`

Text styling properties

- `font-family`: chooses the typeface and fallback fonts.
- `font-size`: controls text size.
- `font-weight`: controls the thickness of text.
- `color`: sets text color.
- `text-align`: aligns inline content within a block.
- `line-height`: controls line spacing.
- `text-decoration`: controls decorations such as underlines.
- `margin`: controls external spacing around text elements.

Example:

```
body {
 font-family: Arial, sans-serif;
}

.heading {
 font-size: 32px;
 font-weight: 700;
}

.muted-text {
 color: #606060;
}
```

Cascade and specificity

When multiple rules affect the same property, the browser resolves them using the CSS cascade, including importance, specificity, and source order. ID selectors are generally more specific than class selectors, which are generally more specific than element selectors. Inspect conflicting rules before reaching for `!important`.

5. CSS Box Model and Padding

The box model describes the space occupied by an element:

1. **Content:** the text, image, or other inner content.
2. **Padding:** space between the content and border.
3. **Border:** the outline around content and padding.

4. **Margin:** space outside the border.

A common reset is:

```
* {
 box-sizing: border-box;
}
```

With `border-box`, declared width and height include padding and border.

Padding-based button sizing

The source exercises include replacing fixed button dimensions with padding. The key idea is to let a button's dimensions respond to its text rather than forcing a fixed width and height.

```
.button {
 padding: 10px 18px;
 border: 0;
 border-radius: 4px;
}
```

The first padding value applies to the top and bottom; the second applies to the left and right. Adjust padding to match the target appearance while preserving other required styles.

Related exercise tasks

- Replace fixed width and height on the referenced buttons with padding.
- Update a Tweet-style button to use padding rather than fixed dimensions.
- Inspect a Subscribe button in Chrome DevTools to retrieve its actual computed background color rather than estimating from a screenshot.

The scan references earlier exercises 3a–3e, but those earlier designs are not included in the uploaded scan. Exact details for those referenced tasks therefore cannot be recovered from this source alone.

6. External Stylesheets and Page Structure

Separating HTML from CSS keeps structure and presentation easier to maintain.

```
<head>
 <meta charset="UTF-8">
 <title>Delivery</title>
```

```
 <link rel="stylesheet" href="styles.css">
</head>
```

The `href` must match the stylesheet filename and relative path.

Example CSS for a centered message:

```
body {
 margin: 0;
 font-family: Arial, sans-serif;
}

.delivery {
 text-align: center;
 padding: 48px 16px;
}

.delivery h1 {
 font-size: 28px;
 font-weight: 600;
}
```

The uploaded exercise scan also contains a design task around the text “Order Online for Touchless Delivery.” The exact font and some visual details are partly obscured in the scan, so the example demonstrates the relevant structure and styling pattern rather than claiming pixel-perfect reproduction.

7. Images and Inputs

Image styling concepts include dimensions, cropping, centering, and rounded corners.

```
.thumbnail {
 width: 200px;
 height: 200px;
 object-fit: cover;
 display: block;
 margin: 0 auto;
}

.rounded-image {
 border-radius: 12px;
}

.circular-image {
```

```
 border-radius: 50%;
}
```

- `object-fit: cover` fills the image box while preserving the image's aspect ratio; excess content may be cropped.
- `border-radius` rounds corners.
- `border-radius: 50%` makes an image circular when the box has equal width and height.
- `display: block` plus auto horizontal margins is a common way to center a fixed-width image.

Example text input:

```
<input type="search" placeholder="Search Google or type a URL">
```

```
input {
 padding: 10px 14px;
 border: 1px solid #ccc;
 border-radius: 6px;
 font-size: 16px;
}
```

Image exercises represented in the scan

- Create a centered 200-pixel square image with rounded corners.
- Apply rounded corners to an image.
- Make an image circular.
- Recreate a search-style layout with a logo above a search input.

The scan's exact radius and some design details are not fully legible, so final visual matching should be checked against the original reference image.

8. Display: Block and Inline-Block

Display behavior determines how elements participate in layout.

- **Block:** normally starts on a new line and takes the available width.
- **Inline:** flows within text and does not behave like a block box for width and height.
- **Inline-block:** flows alongside inline content while still accepting width and height.

Example: stack two text inputs vertically.

```
<input class="text-box" placeholder="First">
<input class="text-box" placeholder="Second">
```

```
.text-box {
 display: block;
 margin-bottom: 8px;
}
```

Example: place text and buttons side by side.

```
.survey p {
 display: inline-block;
 width: 220px;
 vertical-align: middle;
}

.survey button {
 vertical-align: middle;
}
```

A Flexbox layout is a modern alternative when the requirement is primarily alignment and spacing, but the exercises specifically use inline-block to teach its behavior.

9. Div Containers, Borders, Shadows, and Cards

A `div` groups content so it can be styled and positioned as a unit.

Square and circle examples

```
<div class="red-square"></div>
<div class="green-circle"></div>
```

```
.red-square {
 width: 100px;
 height: 100px;
 background: red;
}

.green-circle {
 width: 100px;
 height: 100px;
 background: green;
 border-radius: 50%;
}
```

Borders and shadows

```
.notice {
 border: 1px solid #ccc;
 padding: 16px;
 max-width: 360px;
}

.post {
 padding: 16px;
 border-radius: 8px;
 box-shadow: 0 1px 6px rgba(0, 0, 0, 0.15);
}
```

- `border` draws an outline around an element.
- `border-radius` rounds corners.
- `box-shadow` adds a shadow.
- `padding` provides internal spacing.
- `max-width` limits how wide a component can become.

Profile-card layout pattern

A profile card can contain an image, a name, mutual-friend text, and a button. Repeated cards can be placed side by side with `display: inline-block`, as the exercise requests, or with a parent Grid/Flexbox layout in a modern implementation.

```
.profile-card {
 display: inline-block;
 width: 200px;
 vertical-align: top;
 margin: 8px;
 padding: 12px;
 border: 1px solid #ddd;
 border-radius: 8px;
}
```

10. Nested Layouts

A major practical skill is decomposing a reference design into groups instead of positioning every item individually.

A reliable process is:

5. Identify the largest outer container.
6. Decide whether its immediate children form a row or a column.
7. Group related content together.
8. Repeat the row/column decision inside each group.
9. Add spacing, typography, borders, and color after the structure is clear.

For example, a profile card may contain a horizontal row with an avatar and a vertical text group. The text group may contain a name, subtitle, and metadata. Each group can have its own layout rules.

This nested approach is reusable for cards, social posts, navigation, and video metadata.

11. CSS Grid

Grid is useful for two-dimensional layouts with rows and columns. The parent defines the grid; child elements are placed into its tracks.

Fixed-width columns

```
.grid-a {
 display: grid;
 grid-template-columns: 200px 75px;
}
```

Fixed, flexible, and fixed columns

```
.grid-b {
 display: grid;
 grid-template-columns: 50px 1fr 75px;
}
```

Here, `1fr` uses the remaining available track space after fixed tracks and gaps are accounted for.

Four equal columns with different row/column gaps

```
.grid-c {
 display: grid;
 grid-template-columns: repeat(4, 1fr);
 column-gap: 20px;
 row-gap: 10px;
}
```

- `display: grid` activates Grid layout.
- `grid-template-columns` defines column tracks.
- `repeat(4, 1fr)` creates four equal fractional columns.
- `column-gap` controls horizontal spacing.
- `row-gap` controls vertical spacing.
- `align-items: center` can align items within their grid areas.

The source exercises include fixed columns, a flexible middle column, four equal columns, and the use of Grid in a more complex nested layout.

12. Flexbox

Flexbox is useful for arranging items along a main axis and aligning them along a cross axis.

With the default `flex-direction: row`:

- The main axis is horizontal.
- The cross axis is vertical.
- `justify-content` distributes items along the main axis.
- `align-items` aligns items along the cross axis.

Fixed-width items

```
.flex-a {
 display: flex;
}

.flex-a .item-one {
 width: 200px;
}

.flex-a .item-two {
 width: 75px;
}
```

Fixed, flexible, and fixed items

```
.flex-b {
 display: flex;
}
```

```
.flex-b .left {
 width: 50px;
}

.flex-b .middle {
 flex: 1;
 min-width: 0;
}

.flex-b .right {
 width: 75px;
}
```

`flex: 1` allows the middle item to grow into the available space. `min-width: 0` can prevent long content from forcing that item wider than intended.

Distribute items across a row

```
.flex-c {
 display: flex;
 justify-content: space-between;
}
```

`space-between` puts the first item at the start and the last at the end, distributing remaining space between them.

Align items in a fixed-height box

```
.flex-d {
 height: 50px;
 border: 1px solid gray;
 display: flex;
 align-items: center;
}
```

Vertical navigation

```
.nav {
 display: flex;
 flex-direction: column;
 gap: 12px;
}
```

The scan's Flexbox exercises cover fixed widths, flexible remaining space, evenly distributed items, cross-axis alignment, vertical navigation, and a more complex visual layout.

13. Nested Flexbox

A Flexbox container can be placed inside another Flexbox container. This makes it possible to control the layout of a large component and then independently align the items within each section.

```
<div class="outer">
 <div class="first">
   <span>Item 1</span>
 </div>
 <div class="second">Item 2</div>
</div>
```

```
.outer {
 display: flex;
}

.first {
 width: 100px;
 display: flex;
 align-items: center;
 justify-content: center;
}

.second {
 width: 200px;
}
```

The outer container arranges its two children horizontally. The first child is itself a Flexbox container, so its own content can be centered independently.

The practical lesson is to choose layout behavior at the level of the parent that needs to arrange its children.

14. Fixed Positioning

A fixed element is positioned relative to the viewport and generally remains in place as the page scrolls.

Bottom-right element

```
body {
 height: 3000px;
}

.fixed-corner {
```

```
 position: fixed;
 right: 20px;
 bottom: 20px;
}
```

Fixed sidebar

```
.sidebar {
 position: fixed;
 top: 0;
 right: 0;
 bottom: 0;
 width: 100px;
 background: #eee;
}
```

Full-viewport overlay

```
.overlay {
 position: fixed;
 inset: 0;
 background: black;
}
```

`inset: 0` is shorthand for setting all four offsets—top, right, bottom, and left—to zero. A suitable `z-index` may be needed if the overlay must appear above other content.

15. Absolute and Relative Positioning

An absolutely positioned element is removed from normal document flow and positioned relative to its nearest ancestor that establishes a containing block, commonly an ancestor with `position: relative`.

Close button in a container

```
<div class="positioned-box">
 <button class="close">×</button>
</div>
```

```
.positioned-box {
 position: relative;
}

.close {
 position: absolute;
 top: 0;
```

```
  right: 0;
}
```

The parent establishes the positioning context. The button is placed at the parent's top-right corner.

Fixed video-style box

```
.video-ad {
  position: fixed;
  right: 20px;
  bottom: 20px;
  width: 320px;
  height: 180px;
  background: #ddd;
}
```

Close button partly outside the box

```
.video-ad .close {
  position: absolute;
  top: -12px;
  left: -12px;
}
```

The negative offsets place the button partly outside the box. The `.video-ad` element is fixed relative to the viewport, while the close button is absolute relative to the positioned ad container.

Timestamp in the bottom-right corner

```
<div class="video-ad">
  <button class="close">×</button>
  <span class="timestamp">0:20</span>
</div>
```

```
.timestamp {
  position: absolute;
  right: 6px;
  bottom: 6px;
  color: white;
  background: rgba(0, 0, 0, 0.8);
  padding: 3px 5px;
  font-size: 12px;
}
```

This pattern demonstrates how nested positioning can place multiple small elements at precise corners of a larger component.

16. Chrome DevTools and Debugging

The exercise scan includes inspecting an element's actual styling in Chrome DevTools.

A practical workflow:

10. Right-click the target element and select **Inspect**.
11. Use the Elements panel to identify the relevant HTML node.
12. Use Styles to view applicable CSS rules and temporarily change values.
13. Use Computed to inspect final values such as color, width, height, padding, margin, and font.
14. Click a color swatch to inspect or copy the color value.
15. Check crossed-out declarations for overridden rules.
16. If CSS changes do not appear, check the stylesheet path, file save state, and browser console.

The key skill is to inspect the browser's actual computed value rather than estimating a value from a screenshot.

17. Choosing the Right Layout Technique

Requirement | Appropriate technique

Space inside a component | `padding`

Space outside a component | `margin`

Repeated rows and columns | CSS Grid

Arrange items in one row or column | Flexbox

Basic side-by-side box behavior in the specified exercise | `inline-block`

Pin a sidebar or overlay to the viewport | `position: fixed`

Place a timestamp at a component's corner | `position: absolute` inside a positioned parent

Make an image circular | Equal width/height, `object-fit: cover`, `border-radius: 50%`

Inspect an exact rendered color | Chrome DevTools, especially Computed and Styles